

TEORÍA DE FUNCIONAMIENTO

Para implementar el juego Box Tower en Minecraft de manera funcional, se requiere comprender cómo interactúan los componentes físicos (Arduino) con el entorno virtual (Minecraft). A continuación se detalla el proceso teórico completo:

1. Arquitectura del Sistema

El sistema funciona mediante tres capas principales que se comunican entre sí:

- **Capa Física:** Arduino Uno conectado al push button mediante el protoboard, que detecta las pulsaciones del jugador
- **Capa de Comunicación:** Programa Arduino que traduce las señales físicas en comandos seriales enviados a la computadora
- **Capa Virtual:** Minecraft que recibe los comandos y ejecuta las acciones correspondientes en el mundo del juego

2. Mecánica de la Pantalla (40x65 bloques)

La "pantalla" de juego está construida de la siguiente manera:

- **Estructura base:** 40 columnas verticales de 65 bloques de altura cada una, construidas con concreto blanco
- **Sistema de iluminación:** Cada columna tiene internamente bloques de Linterna Marina que se activan mediante circuitos de Redstone
- **Pixel activo:** Cuando una posición debe "encenderse", el circuito de Redstone activa la Linterna Marina correspondiente, haciéndola visible a través del concreto blanco

3. Circuito de Redstone - Funcionamiento Detallado

El sistema de Redstone controla cada columna de manera individual:

1. **Distribución vertical:** Cada columna tiene 65 linternas marinas apiladas verticalmente, cada una conectada a un circuito independiente
2. **Repetidores:** Se utilizan repetidores de Redstone cada 15 bloques para mantener la señal fuerte a lo largo de toda la altura (ya que la señal de Redstone se debilita después de 15 bloques)
3. **Sistema de direccionamiento:** Un circuito central de polvo de Redstone funciona como "bus de datos" que puede activar cualquier combinación de las 40 columnas
4. **Activación simultánea:** Para formar una figura (como un bloque cayendo), múltiples circuitos se activan al mismo tiempo, iluminando las linternas correspondientes

4. Proceso de Comunicación Arduino-Minecraft

La secuencia de eventos cuando el jugador presiona el botón:

1. **Detección física:** El push button 12x12 cierra el circuito en el protoboard cuando se presiona
2. **Lectura digital:** Arduino detecta el cambio de estado (de LOW a HIGH) en el pin configurado como INPUT
3. **Procesamiento:** El programa Arduino valida que no sea un rebote (debouncing) y confirma la pulsación legítima
4. **Transmisión serial:** Arduino envía un comando a través del puerto serial USB (ejemplo: "BUTTON_PRESS\n")
5. **Recepción en Minecraft:** Un plugin o mod (como Raspberry Jam Mod o similar) escucha el puerto serial y recibe el comando
6. **Ejecución de comando:** El plugin traduce el comando en una acción de Minecraft, específicamente activando el circuito de Redstone correspondiente

5. Lógica del Juego en Minecraft

La mecánica del juego Box Tower funciona mediante:

- **Bloque en movimiento:** Un grupo de linternas se activa en secuencia horizontal (de izquierda a derecha) simulando un bloque que se desplaza
- **Velocidad variable:** La velocidad del desplazamiento se controla mediante el delay entre activaciones sucesivas de columnas adyacentes
- **Detección de pulsación:** Cuando Arduino envía la señal, el sistema registra en qué posición horizontal estaba el bloque en ese momento
- **Cálculo de precisión:** Se compara la posición de pulsación con la posición del bloque anterior en la torre
- **Recorte progresivo:** Si hay desalineación, el nuevo bloque se recorta, reduciendo el ancho disponible para el siguiente nivel
- **Visualización de la torre:** Los bloques colocados permanecen iluminados en sus posiciones verticales correspondientes, formando la torre

6. Sistema de Coordenadas y Posicionamiento

Para controlar con precisión cada pixel de la pantalla:

- **Coordenada X:** Representa la columna horizontal (1 a 40)
- **Coordenada Y:** Representa la altura vertical (1 a 65)
- **Matriz de estado:** El programa mantiene un array bidimensional [40][65] que registra qué linternas deben estar encendidas
- **Actualización continua:** Cada frame del juego actualiza la matriz y envía comandos de Redstone para reflejar el estado actual

7. Optimización de Recursos de Redstone

Dado que se requieren 2600 bloques de conexión, la optimización es crucial:

- **Multiplexación:** En lugar de un cable por cada linterna, se pueden usar circuitos multiplexados que activen filas y columnas

- **Comandos de bloque:** Alternativamente, usar command blocks con /setblock para cambiar directamente bloques luminosos sin Redstone
- **Estructuras repetitivas:** Diseñar un módulo base de 5x5 o 10x10 que se pueda clonar usando /clone para acelerar la construcción

8. Sincronización y Timing

La sincronización precisa es fundamental para una jugabilidad fluida:

- **Tick rate de Minecraft:** Minecraft opera a 20 ticks por segundo, lo que significa actualizaciones cada 50ms
- **Latencia serial:** La comunicación serial Arduino-PC típicamente tiene ~10ms de latencia
- **Buffer de comandos:** Implementar un buffer que acumule comandos y los ejecute en el siguiente tick disponible
- **Feedback visual:** Asegurar que el jugador vea la respuesta en menos de 100ms para mantener la sensación de respuesta inmediata

9. Alternativas de Implementación

Existen diferentes enfoques técnicos según las capacidades disponibles:

- **Método A - Redstone puro:** Usa solo mecánicas de Redstone vanilla, más laborioso pero funciona sin mods
- **Método B - Command Blocks:** Utiliza bloques de comando para controlar directamente los bloques, más eficiente pero requiere permisos de operador
- **Método C - Mods/Plugins:** Usa Raspberry Jam, ComputerCraft o mods similares para comunicación directa Python/Java, más flexible y escalable